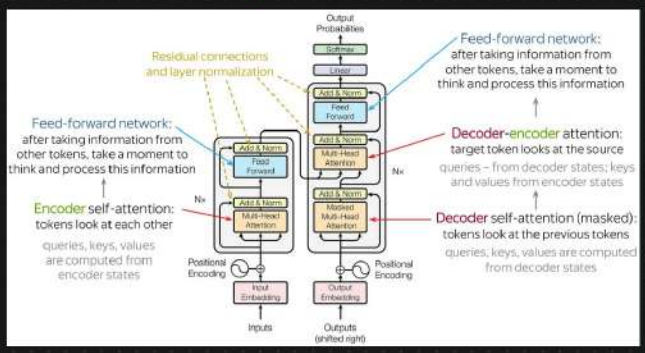
Introduction To Transformers

1) RNN / LSTM / GRU RNN
2) Encoder Decoder Architecture
3) ATTENTION MECHANISM
4) TRANSFORMERS
   1. Why Transformers?
   2. Architecture of Transformers?
   3. SELF ATTENTION → Q, K, V
   4. Positional Encoding
   5. Multi Head ATTENTION
   6. Combining the working of Transformers

Architecture



Generative AI → LLM, Multimodel

BERT, GPT ←

↓

Open AI → Chat GPT

↓

GPT-4o

1) What And Why → Transformers

Transformers in natural language processing (NLP) are a type of deep learning model that use self-attention mechanisms to analyze and process natural language data. They are encoder-decoder models that can be used for many applications, including machine translation. ⇒ Seq2Seq Task

Eg: Language Translation → Google Translation

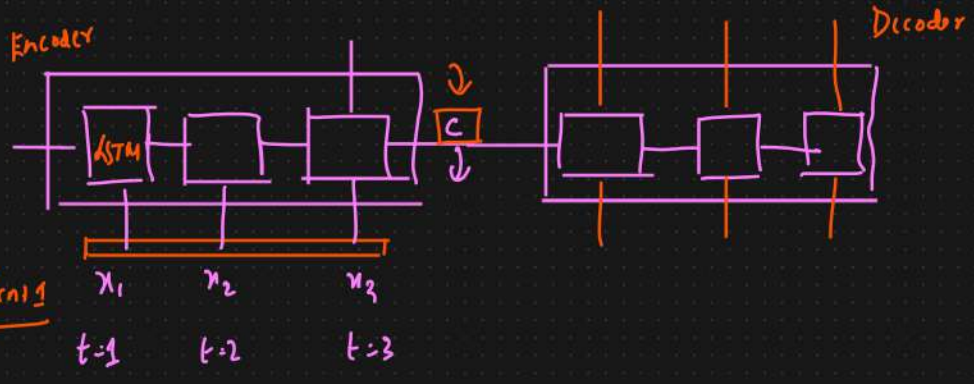
English → French

i/p ⇒ Many → o/p: Many. {Length of the sentence}.

Encoder - Decoder

Sentence Length ↑↑ → Bleau Score ↓↓

Length Sentence ↑↑.



3.1 Decoder: General Description

In a new model architecture, we define each conditional probability in Eq. (2) as:

$$p(y_i|y_1,\dots,y_{i-1},\mathbf{x}) = g(y_{i-1}, s_i, c_i), \quad (4)$$

where s_i is an RNN hidden state for time i, computed by

$$s_i = f(s_{i-1}, y_{i-1}, c_i).$$

It should be noted that unlike the existing encoder–decoder approach (see Eq. (2)), here the probability is conditioned on a distinct context vector c_i for each target word y_i.

The context vector c_i depends on a sequence of *annotations* $(h_1,\cdots,h_{T_x})$ to which an encoder maps the input sentence. Each annotation h_i contains information about the whole input sequence with a strong focus on the parts surrounding the i-th word of the input sequence. We explain in detail how the annotations are computed in the next section.

The context vector c_i is, then, computed as a weighted sum of these annotations h_i:

$$c_i = \sum_{j=1}^{T_x} \alpha_{ij} h_j. \quad (5)$$

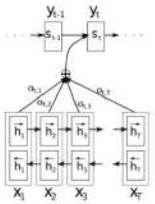
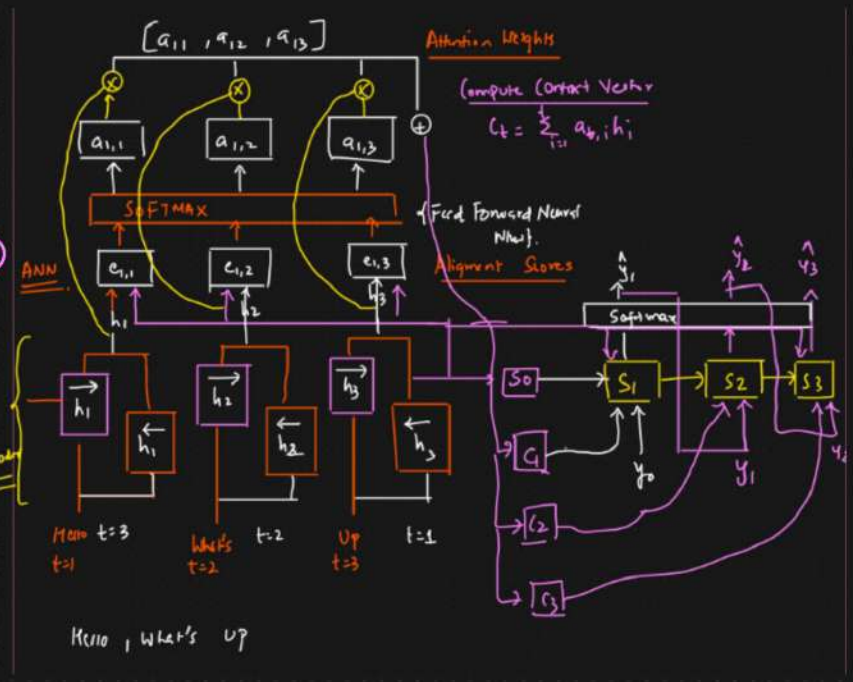


Figure 1: The graphical illustration of the proposed model trying to generate the t-th target word y_t given a source sentence $(x_1, x_2, \dots, x_T)$.



Additional Context ⟶ Decoder

Long Sentence · Accuracy ↑↑.

Attention Mechanism ↓

① Parallely we cannot send all the words in a sentence ⟶ **Scalable**

DATASET ⟶ Huge ⟶ Scalable With Respect to Training.

TRANSFORMERS ≠ LSTM RNN

↳ Self Attention Module ← All the words will be parallely sent to encoder.

↓

Positional Encoding

Transformer ↑↑ DATASET ⟶ Amazing SOTA ← NLP

Transfer Learning ⟶ MultiModal Task ⟶ NLP + Image ←

Transformers ∴ AI Space ⟶ SOTA Model ⟶

↓ ↓

BERT GPT ⟶ Transfer Learning ⟶ SOTA Models ⟶ DALL·E } LLM's Generative AI

↑

Train Huge Data

① Contextual Embedding → Self Attention

Contextual Vectors

Eg: My name is KRISH And I play CRICKET

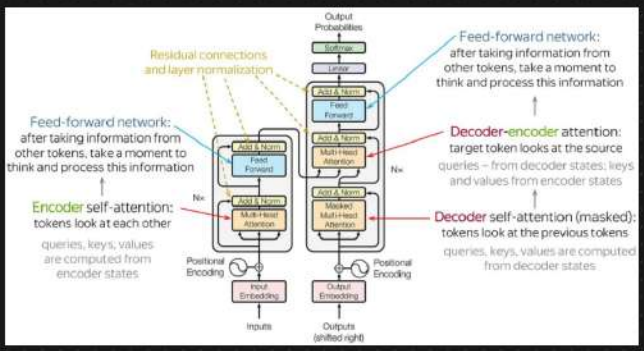
Word2Vec

Embedding Layer

Contextual vector Embedding

② Basic Transformer Architecture

{Seq2Seq Task} → Language Translation {Eng → French}



I/p → The TRANSFORMER → O/p.

6 Encoder

Encoder

Comment vas-tu ?

Encoder → Decoder

HOW ARE YOU?

O/p Encoder Decoder Architecture

← 6 Decoders

Encoder

Feed Forward Neural N/W

Self - Attention

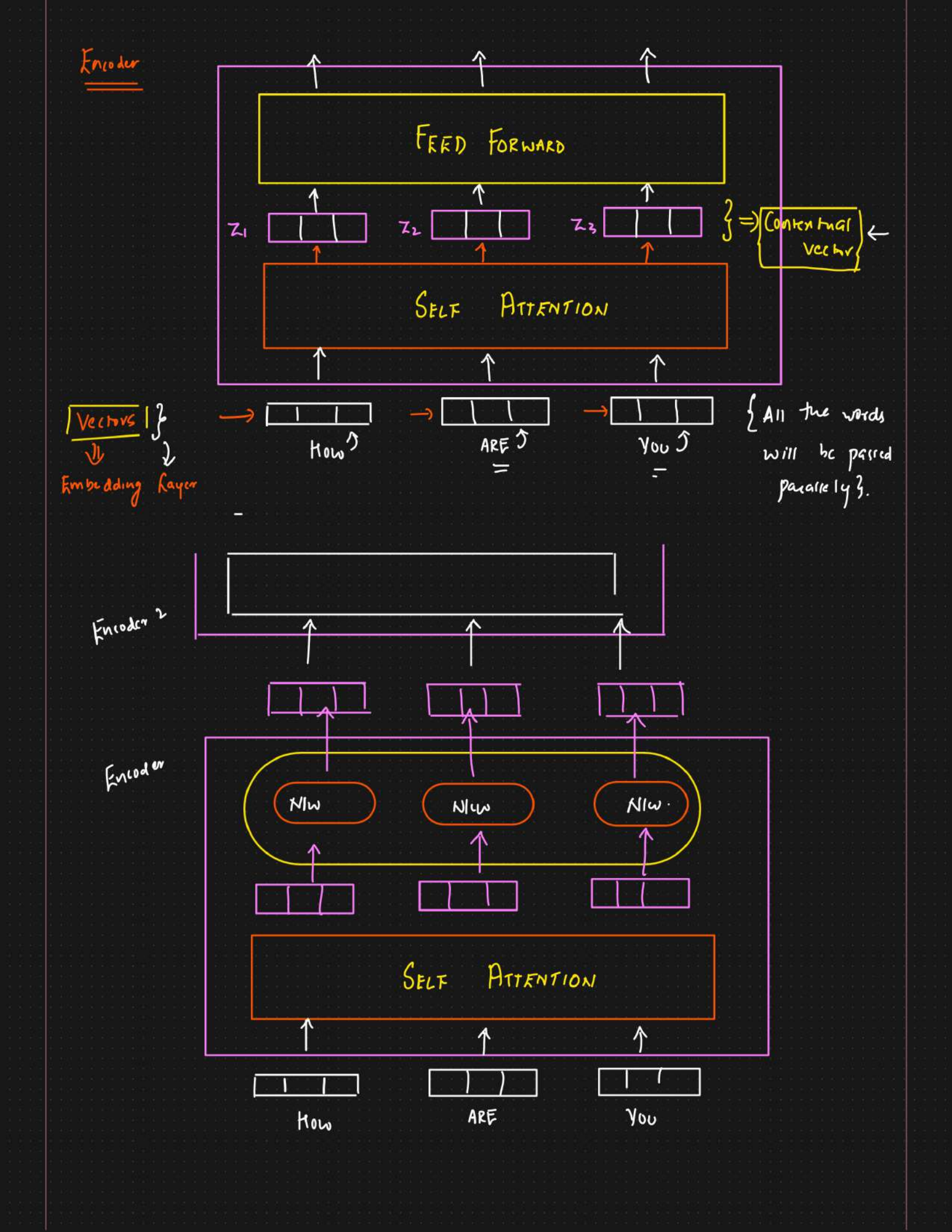
{LSTM RNN

Decoder

Feed Forward

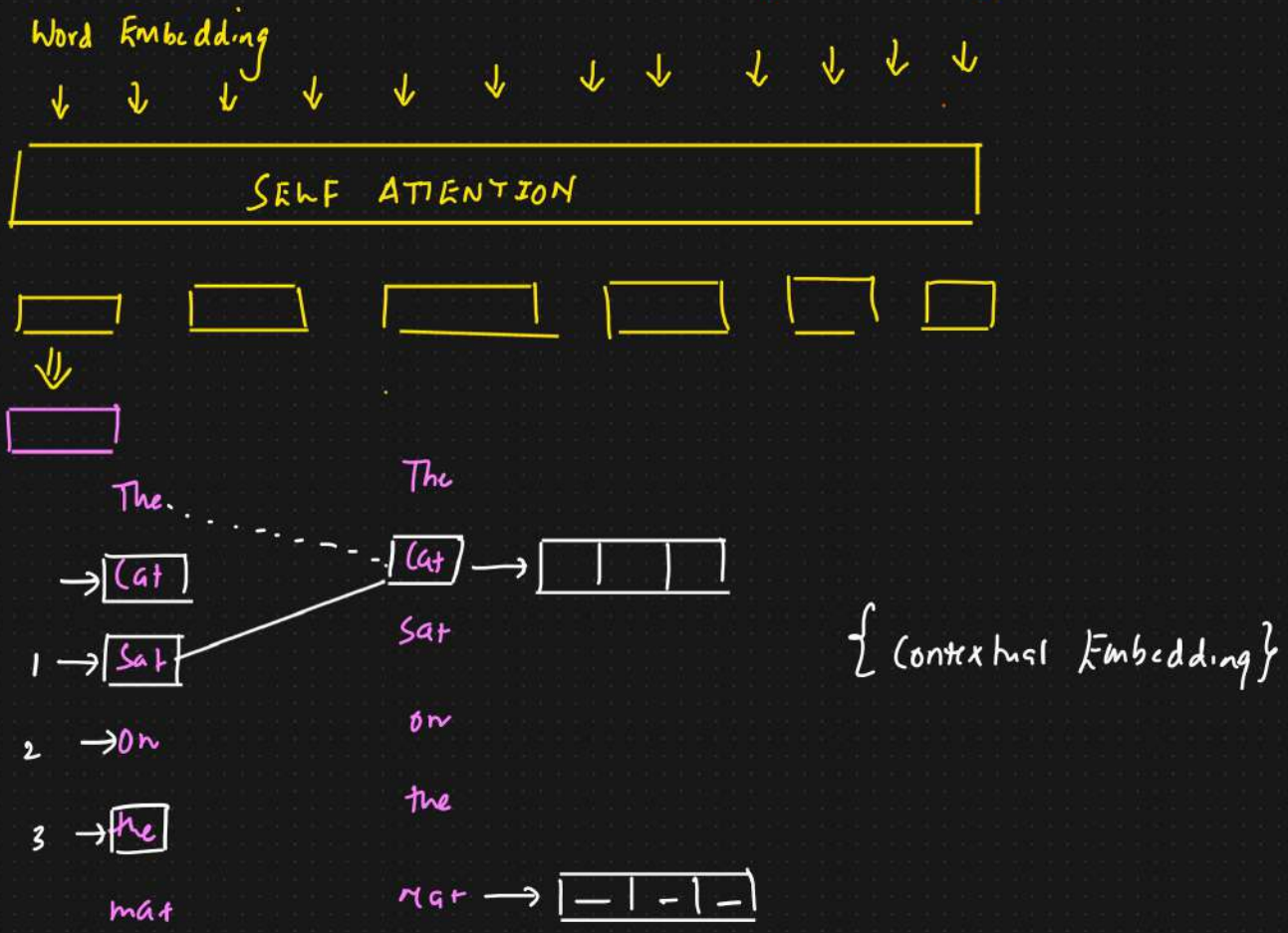
Encoder - Decoder Attention

Self - Attention


Encoder
FEED FORWARD
z_1
z_2
z_3
Contextual vector
SELF ATTENTION
Vectors
Embedding Layer
How
ARE
You
{All the words will be passed parallely}.
Encoder 2
Encoder
NN
NN
NN
SELF ATTENTION
How
ARE
You

Self Attention At a Higher Level

Eg: The cat sat on the mat, the cat lay on the rug.



Self Attention In Detail

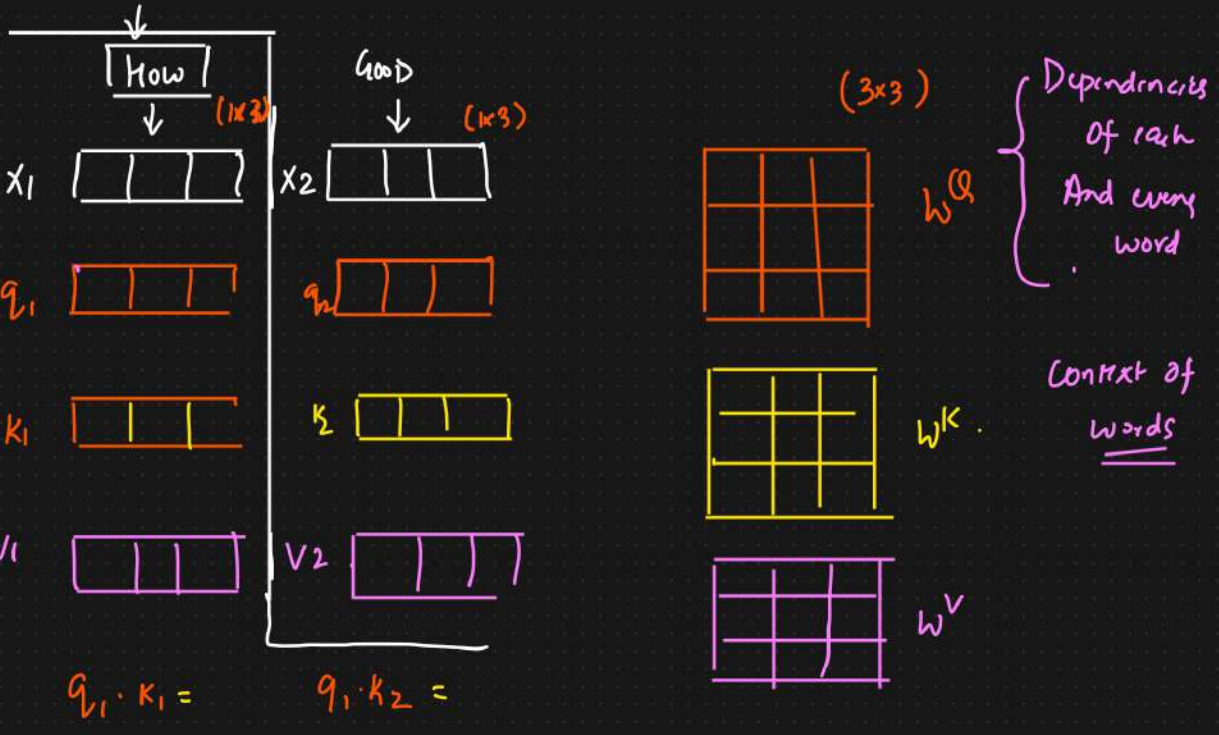
① To create 3 vectors from each of the encoder i/p. Query vector, Key vector, value vector. ⇒ Contextual Embedding

Eg: YT ⇒ Search keywords (Topic's) ⇒ { Query }.

Query ⟶ { Key } → Tags, Description, Title ⇒ Value ⇒ O/P Video

② Second step in calculating self attention is to calculate the score.

The Score determines how much focus to place on the other part of the sentence.



(*) How much Focus to place on other part of the i/p sentence as we encode a word at certain position

Divide $\sqrt{dk}$ — —

Dimension=3 ↓ Variance

Dimension ↑↑↑ T12 O/p ↑↑↑ Variance ↑↑↑ ⇒

$\frac{\text{Variance}}{\sqrt{dk}} \approx \frac{\text{Variance}\uparrow\uparrow\uparrow}{\sqrt{dk}}$

{ More stable gradients }

Softmax [0-1] [0-1].

0.88 + 0.12 = 1

Softmax => The Softmax score determines how much each word will be expressed at this position

X X X

Value Vectors V1 V2.

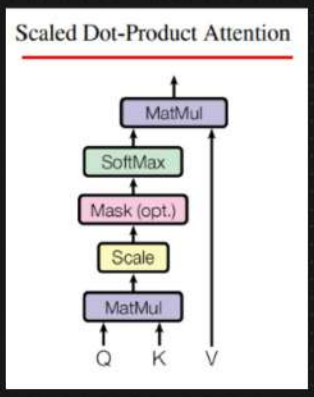
0.88 | 1 | 1 | 1 |

| 0.88 | 0.88 | 0.88 | + | 0.12 | 0.6 | 0.12 |

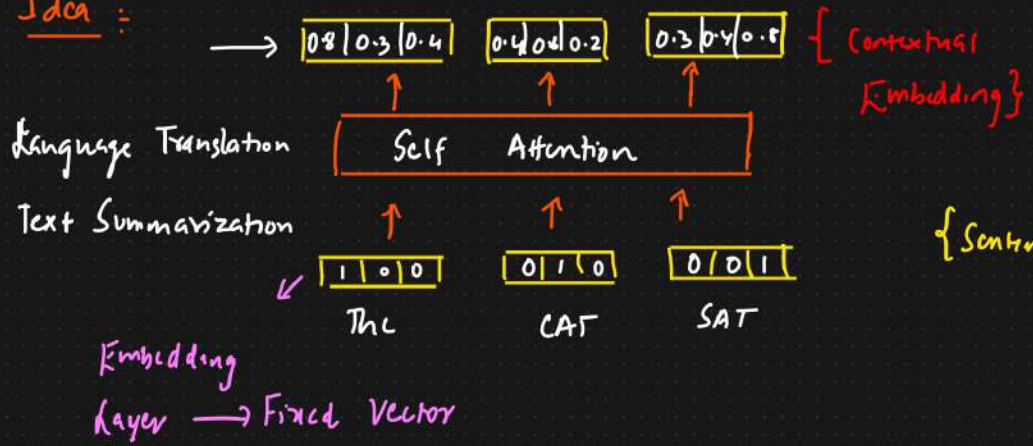
Contextual vector → Z1 Z2

Self Attention At Higher And Detailed Level

Self-attention, also known as scaled dot-product attention, is a crucial mechanism in the transformer architecture that allows the model to weigh the importance of different tokens in the input sequence relative to each other



Idea :



{ Sentence, Dataset }.

Embedding Layer ⟶ Fixed Vector

1) Inputs : Queries, Keys, And Values

Model ⟶ Queries, Keys And Values

1. Query Vectors (Q):
Role: Query vectors represent the token for which we are calculating the attention. They help determine the importance of other tokens in the context of the current token.

Importance:

Focus Determination: **Queries help the model decide which parts of the sequence to focus on for each specific token. By calculating the dot product between a query vector and all key vectors, the model assesses how much attention to give to each token relative to the current token.**

Contextual Understanding: **Queries contribute to understanding the relationship between the current token and the rest of the sequence, which is essential for capturing dependencies and context.**

2. Key Vectors (K):
Role: Key vectors represent all the tokens in the sequence and are used to compare with the query vectors to calculate attention scores.

Importance:

Relevance Measurement: **Keys are compared with queries to measure the relevance or compatibility of each token with the current token. This comparison helps in determining how much attention each token should receive.**
Information Retrieval: **Keys play a critical role in retrieving the most relevant information from the sequence by providing a basis for the attention mechanism to compute similarity scores.**

3. Value Vectors (V):
Role: Value vectors hold the actual information that will be aggregated to form the output of the attention mechanism.

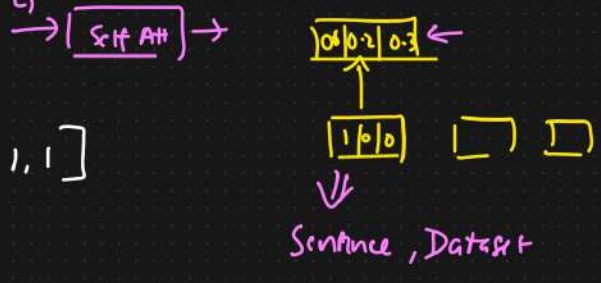
Importance:

Information Aggregation: Values contain the data that will be weighted by the attention scores. The weighted sum of values forms the output of the self-attention mechanism, which is then passed on to the next layers in the network.
Context Preservation: By weighting the values according to the attention scores, the model preserves and aggregates relevant context from the entire sequence, which is crucial for tasks like translation, summarization, and more.

Input Sequence = ["The", "CAT", "SAT"]

Embedding Size = 4

Q, K, V ⇒ 4



① Token Embedding

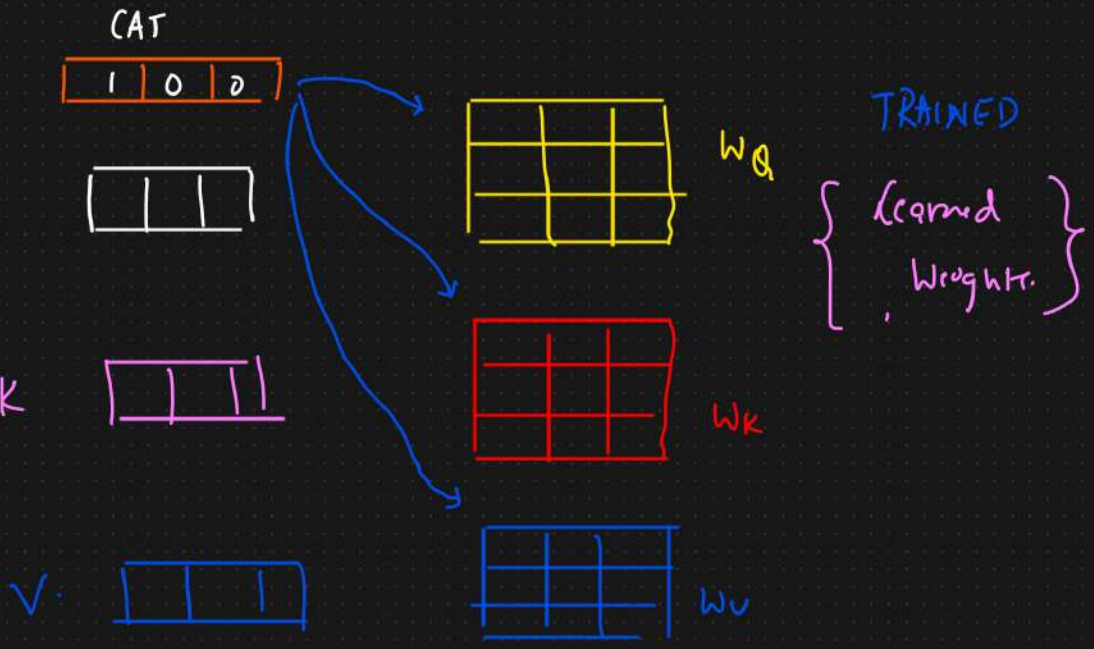
$E_{The} = [1\ 0\ 1\ 0]$

$E_{CAT} = [0\ 1\ 0\ 1]$

$E_{Sat} = [1, 1, 1, 1]$

② Linear Transformation

We create Q, K, V by multiplying the embeddings by learned weights matrices W_Q, W_K and W_V.



Lets consider

$W_Q = W_K = W_V = I$

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \leftarrow$$

$$Q_{The} = [1\ 0\ 1\ 0] \cdot \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = [1\ 0\ 1\ 0]$$

$$K_{The} = [1\ 0\ 1\ 0]$$

$$V_{The} = [1\ 0\ 1\ 0]$$

① $Q_{The} = K_{The} = V_{The} = [1\ 0\ 1\ 0]$

② $Q_{CAT} = K_{CAT} = V_{CAT} = [0\ 1\ 0\ 1]$

③ $Q_{SAT} = K_{SAT} = V_{SAT} = [1,1,1,1]$

③ Compute Attention Scores

$[1\,0\,1\,0]\begin{bmatrix}1\\0\\1\\0\end{bmatrix}$

The

$\text{Score}(Q_{The}, K_{The}) = [1\,0\,1\,0] \cdot [1\,0\,1\,0]^T = 2$

$\text{Score}(Q_{The}, K_{CAT}) = [1\,0\,1\,0] \cdot [0\,1\,0\,1]^T = 0$

$\text{Score}(Q_{The}, K_{SAT}) = [1\,0\,1\,0] \cdot [1,1,1,1]^T = 2$

For the token CAT

$\text{Score}(Q_{CAT}, K_{The}) = [0\,1\,0\,1] \cdot [1\,0\,1\,0]^T = 0$

$\text{Score}(Q_{CAT}, K_{CAT}) = [0\,1\,0\,1] \cdot [0\,1\,0\,1]^T = 2$

$\text{Score}(Q_{CAT}, K_{SAT}) = [0\,1\,0\,1] \cdot [1\,1\,1\,1]^T = 2.$

For the Token SAT

$\text{Score}(Q_{SAT}, K_{The}) = [1,1,1,1] \cdot [1\,0\,1\,0]^T = 2$

$\text{Score}(Q_{SAT}, K_{CAT}) = 2$

$\text{Score}(Q_{SAT}, K_{SAT}) = 4$

④ Scaling ÷

We take up the scores and scale down by dividing the scores by the $\sqrt{d_k}$ $\Rightarrow$ $d_k = 4$ $\quad \sqrt{d_k} = 2.$

Scaling in the attention mechanism is crucial to prevent the dot product from growing too large. $\Rightarrow$ Ensure stable gradients during Training.

d_k is large $\longrightarrow$

① Gradient Exploding

② Softmax Saturation $\{$ $\nearrow$ $\searrow$ $\}$ $\rightarrow$ Vanishing Gradient Problem.

$Q = [2\ 3\ 4\ 1]$ $K_1 = [1\ 0\ 1\ 0]$ $K_2 = [0\ 1\ 0\ 1]$

Without Scaling

$Q \cdot K_1^T = 2\times1 + 3\times0 + 4\times1 + 1\times0 = 2 + 4 = 6.$

$Q \cdot K_2^T = 2\times0 + 3\times1 + 4\times0 + 1\times1 = 0 + 3 + 0 + 1 = 4$

*) Score $[6, 4]$ ⇒ Scaling Not Applied

$$\text{Softmax}([6,4]) = \left[\frac{e^6}{e^6+e^4}, \frac{e^4}{e^6+e^4}\right] = \left[\frac{e^6}{e^6(1+e^{-2})}, \frac{e^4}{e^4(e^2+1)}\right]$$

$$= \left[\frac{1}{(1+e^{-2})}, \frac{1}{(e^2+1)}\right]$$

$\approx [0.88, 0.12]$.

(*) Property of Softmax W_Q, W_K, W_V

$([10, 1]) = [0.99, 0.01]$

Dot product = Large values

Most of the attention weight is assigned to the first key vector, very little to the second vector,

With Scaling

$\sqrt{d_k} = \sqrt{4} = \sqrt{2}$ ⇒ Variance 2, 3

dimension ↑ Variance ↑

① Compute Scaled Dot Product

$[6, 4] \Rightarrow \text{Scale} \Rightarrow \left[\frac{6}{2}, \frac{4}{2}\right] = [3, 2]$.

$\sqrt{d_k}$ ↑ same

$$\text{Softmax}([3,2]) = \left[\frac{e^3}{e^3+e^2}, \frac{e^2}{e^3+e^2}\right] = \left[\frac{e^3}{e^3(1+e^{-1})}, \frac{e^2}{e^2(e^1+1)}\right]$$

$= [0.73, 0.27]$ ⇒ Attention Weights

(*) Here, the attention weights are more balanced compared to the unscaled case

Summary of Importance
Stabilizing Training: Scaling prevents extremely large dot products, which helps in stabilizing the gradients during backpropagation, making the training process more stable and efficient.

Preventing Saturation: By scaling the dot products, the softmax function produces more balanced attention weights, preventing the model from focusing too heavily on a single token and ignoring others.
Improved Learning: Balanced attention weights enable the model to learn better representations by considering multiple relevant tokens in the sequence, leading to better performance on tasks that require context understanding.

Scaling ensures that the dot products are kept within a range that allows the softmax function to operate effectively, providing a more balanced distribution of attention weights and improving the overall learning process of the model.

(4) Scaling $= \sqrt{d_k} = \sqrt{4} \Rightarrow 2$

Similarly scaling will be done for all other Tokens.

$\text{Scaled\_score}(Q_{The}, K_{The}) = 2/2 = 1$

$\text{Scaled\_score}(Q_{The}, K_{CAT}) = 0/2 = 0$

$\text{Scaled\_score}(Q_{The}, K_{SAT}) = 2/2 = 1$

(5) Apply Softmax

ATTENTION WEIGHTS "The" $= \text{Softmax}([1,0,1]) = [0.4223, 0.1554, 0.4223]$

ATTENTION WEIGHTS "CAT" $= \text{Softmax}([0,2,2]) = [0.1554, 0.4223, 0.4223]$

ATTENTION WEIGHTS "SAT" $= \text{Softmax}([2,2,4]) = [0.2119, 0.2119, 0.5762]$.

(6) Weight Sum of Values

We multiply the attention weights by corresponding value vectors

For the Token The =

$$\text{Output}_{(\text{The})} = 0.4223 * V_{\text{The}} + 0.1554 * V_{\text{CAT}} + 0.4223 \cdot V_{\text{Sat}}$$

$$= 0.4223\,[1\ 0\ 1\ 0] + 0.1554 * [0\ 1\ 0\ 1] + 0.4223 * [1\ 1\ 1\ 1]$$

$$= [0.4223, 0, 0.4223, 0] + [0, 0.1554, 0, 0.1554] + [0.4223, 0.4223, 0.4223, 0.4223]$$

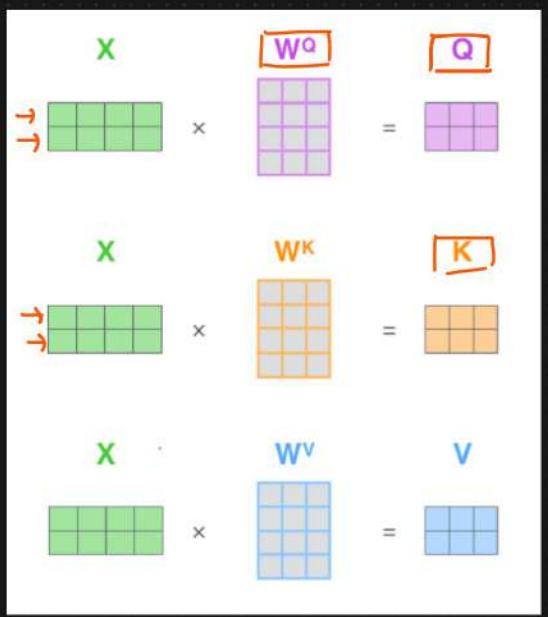
$$= [1.2669, 0.9999, 1.2669, 0.9999].$$

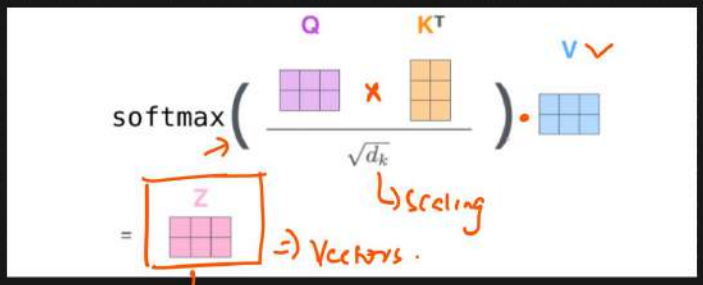
↓ Contextual vector

The [1 0 1 0] ⇒ Self Attention ⇒ [1.2669, 0.9999, 1.2669, 0.9999].

① ↳ Q, K, V [W^Q, W^K, W^V]
② ↳ Attention Score
③ ↳ Scaled
④ ↳ Softmax
⑤ ↳ Weighted Sum of Value (Softmax × V)

⑥ Multi Head Attention

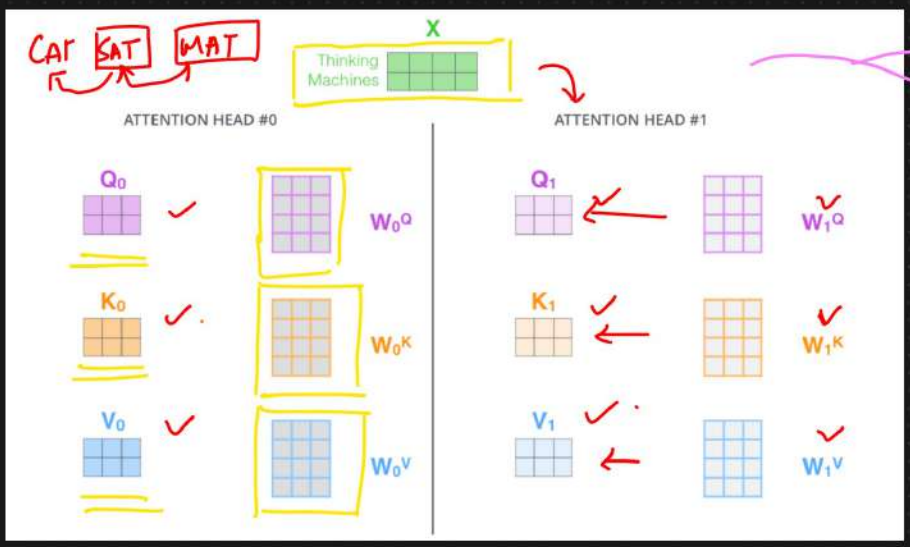




↳ Scaling

Attention Head.

→ Self Attention with Multi Heads



It expands model's ability to focus on different position of tokens.

Score, Softmax X V

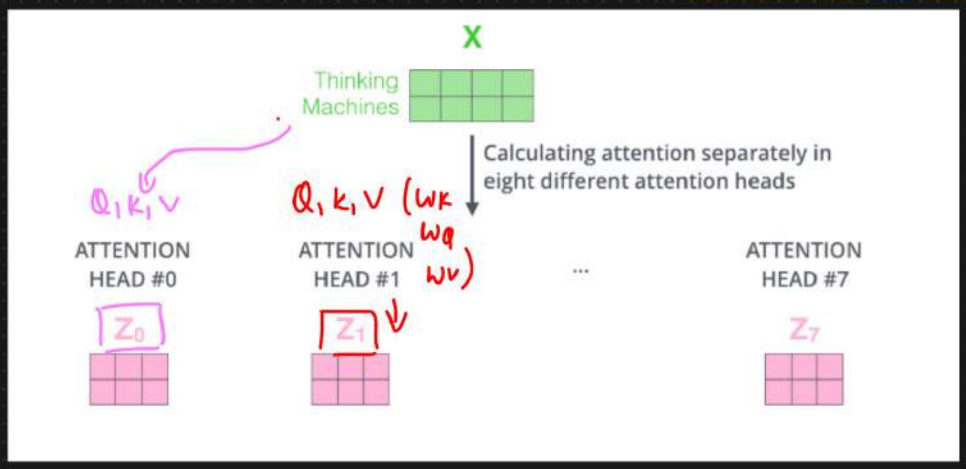
[_ _ _] → Vectors

Z0

[_ _ _]

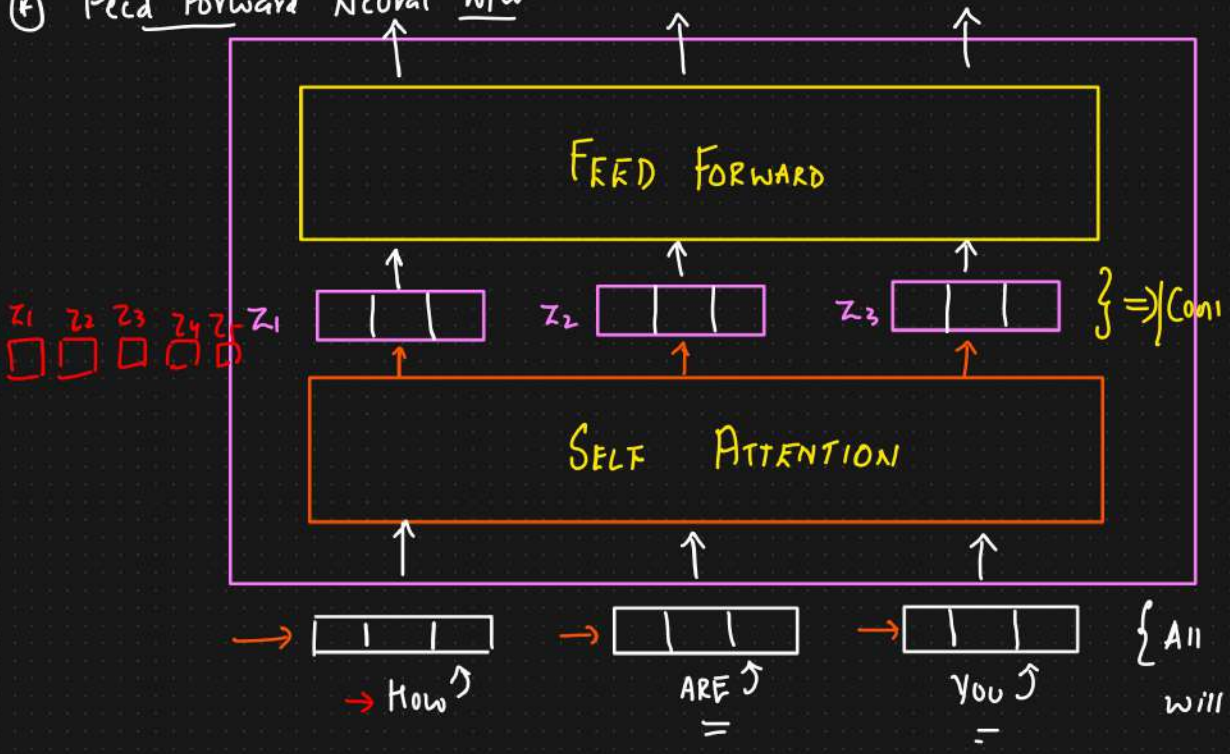
Z1

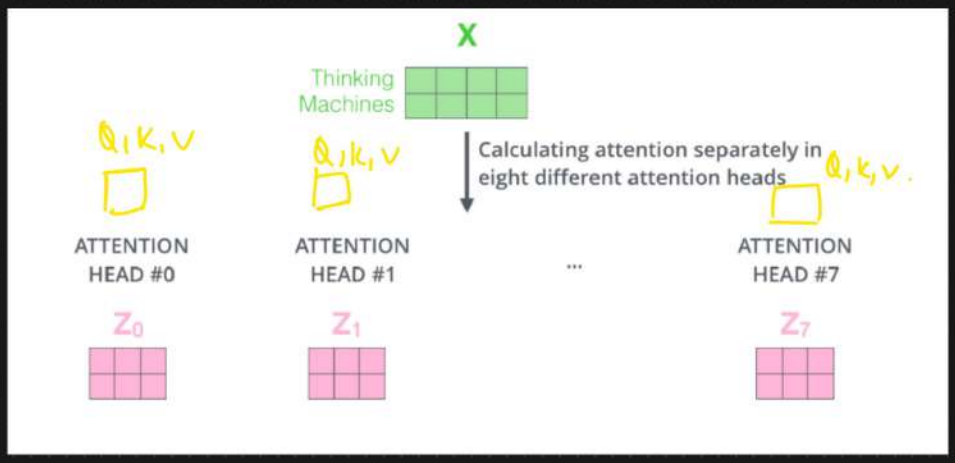
Multi Head Attention

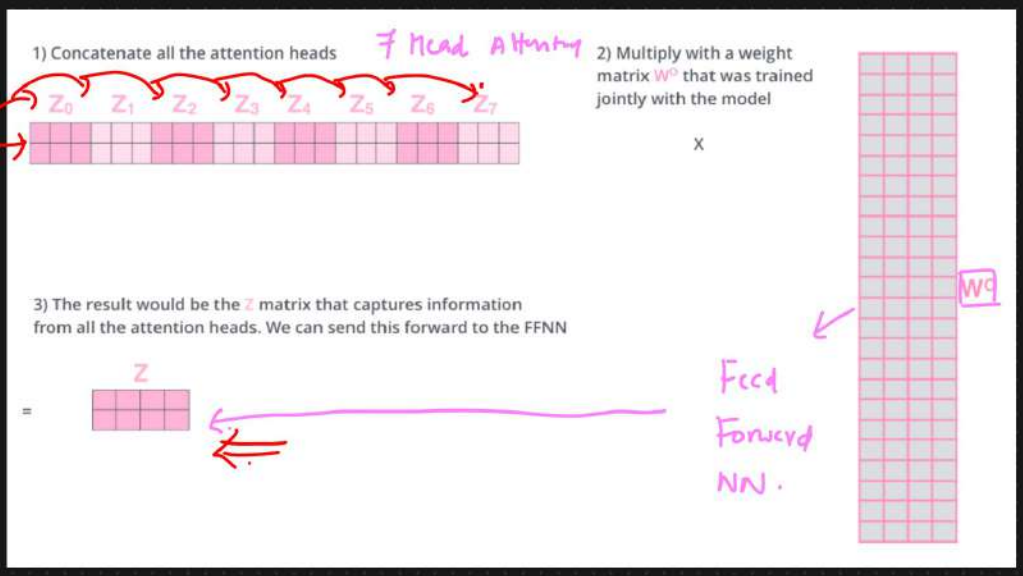


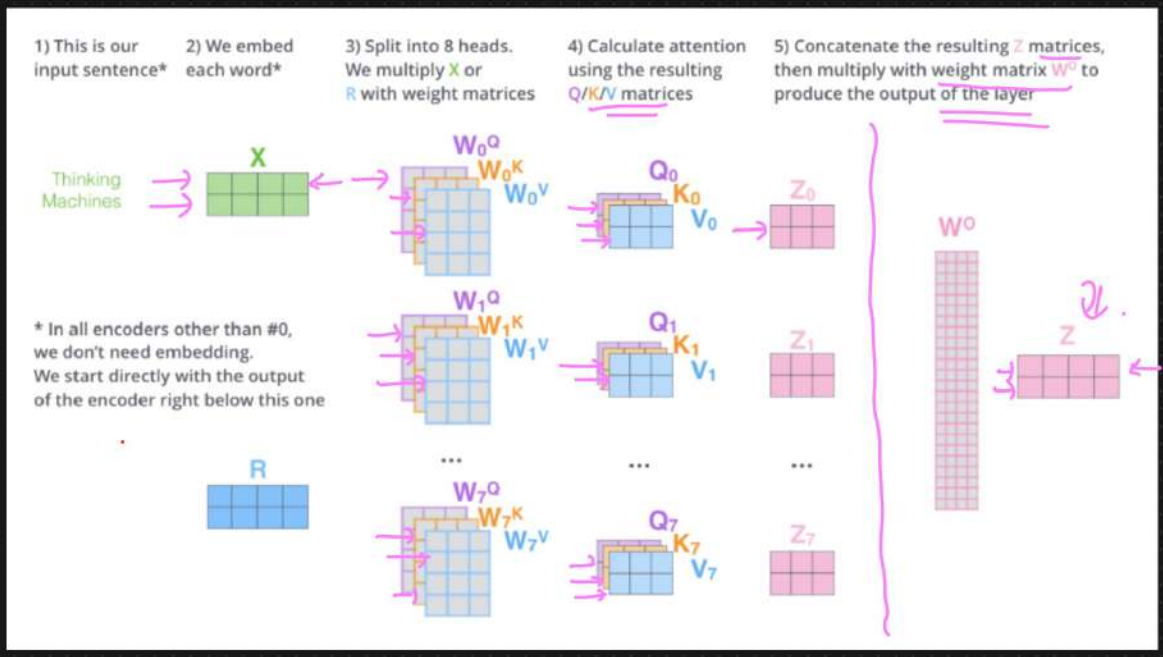
Z Z Z

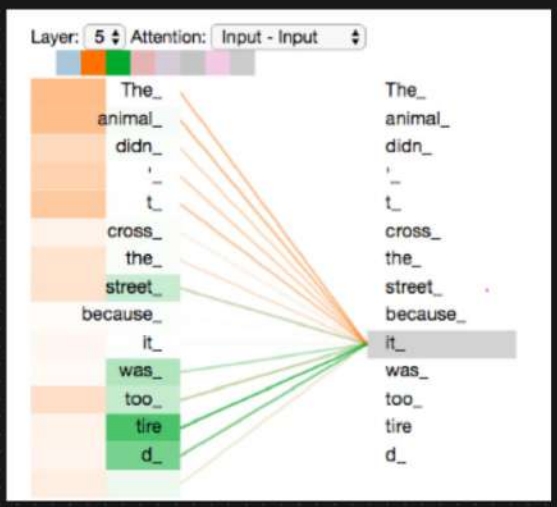
(6) Feed Forward Neural NW

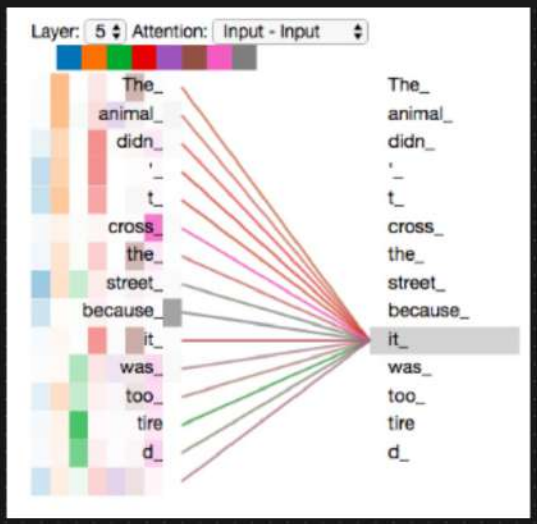










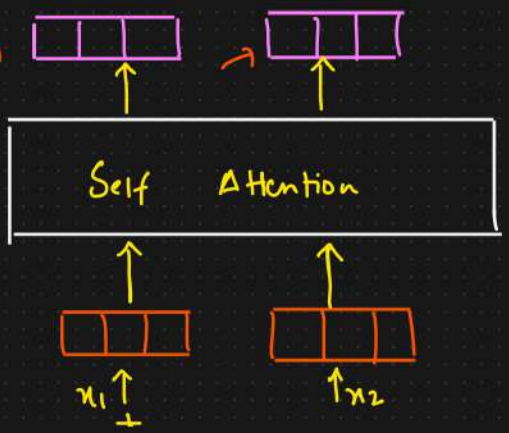


⑥ Positional Encoding – Representing Order of Sequence

① Lion KILLS TIGER
② TIGER KILLS LION

⇓

Missing



Advantage

① Word Tokens it can process parallelly

⇓

DRAW BACK

Lack the sequential structure of the words {order}

Attention Is All You Need

Positional Encoded Vectors

Book, Journal, Novel ↳ 1 lakh words }.

⇓ Backpropagation

Types of Position Encoding

✓1) Sinusoidal Position Encoding ⟶

2) Learned positional Encoding ⇒ Positional Encoding are learned during Training ⟵

① Sinusoidal Positional Encoding : It uses sine and cosine functions of different frequencies to create positional encodings

Formula :

$$P.E(pos, 2i) = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

$$P.E(pos, 2i+1) = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right).$$

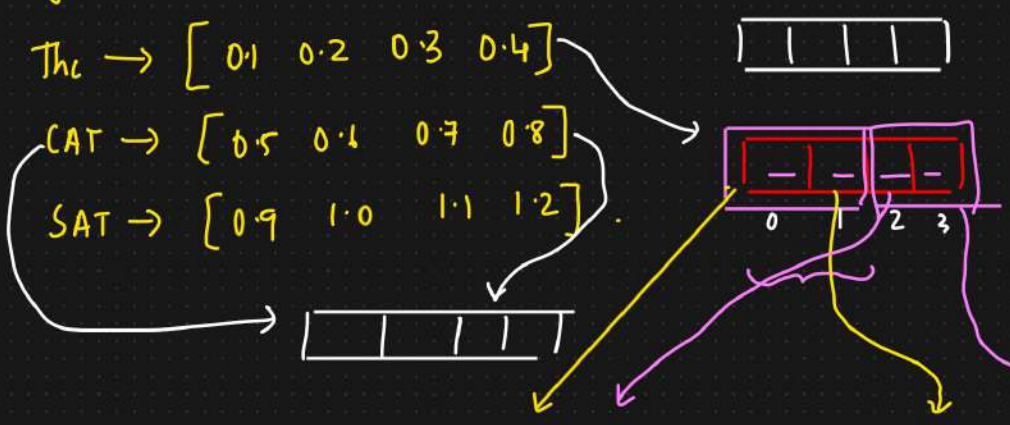
Where pos is the position
i is the dimension
d_{model} is the dimensionality of the embeddings.

Eg: The Cat Sat

The → [0.1 0.2 0.3 0.4]

CAT → [0.5 0.6 0.7 0.8]

SAT → [0.9 1.0 1.1 1.2].



{ Miss the order of the elements }

$$P.E_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \qquad P.E_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

For our example $d_{model} = 4$

For position $pos = 0$

$$P.E(0,0) = \sin\left(\frac{0}{10000^{0/4}}\right) = \sin(0) = 0$$

$$P.E(0,1) = \cos\left(\frac{0}{-}\right) = \cos(0) = 1$$

$$P.E(0,2) = \sin\left(\frac{0}{10000^{2/4}}\right) = \sin(0) = 0$$

$$P.E(0,3) = \cos(0) = 1$$

$P.E = [0, 1, 0, 1]$

$$P.E_{(pos,2i)} = \sin\left(\frac{pos}{10000^{2i/d_{model}}}\right) \qquad P.E_{(pos,2i+1)} = \cos\left(\frac{pos}{10000^{2i/d_{model}}}\right)$$

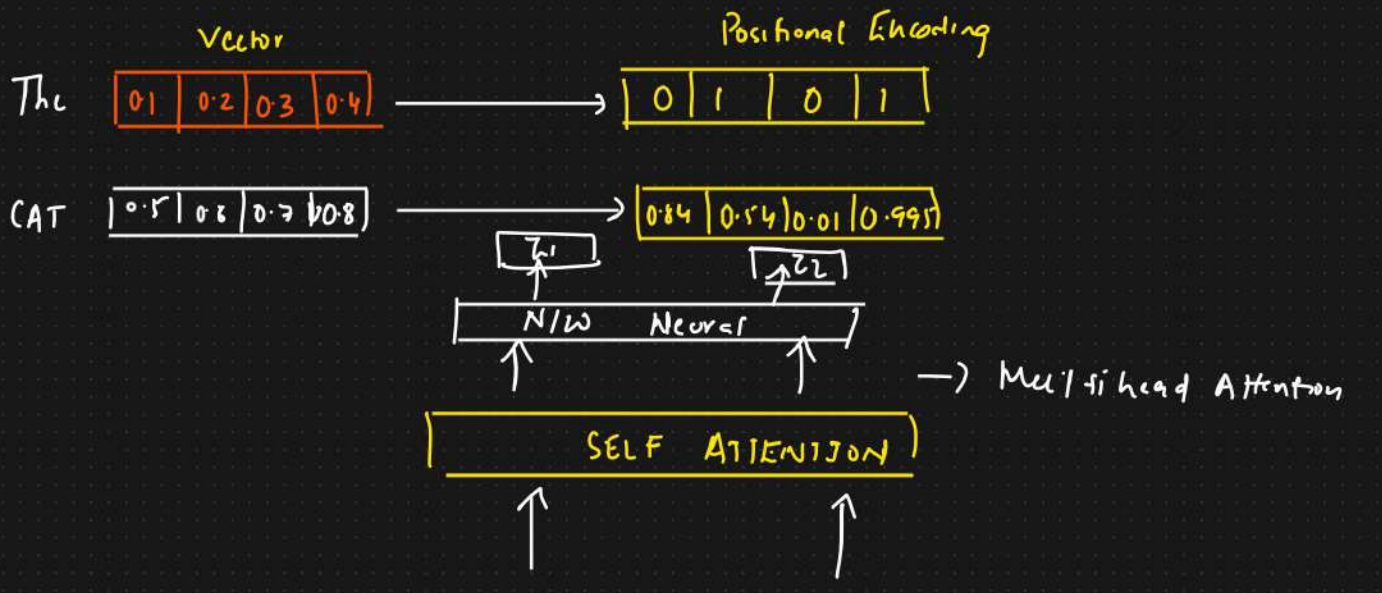
For $pos = 1$

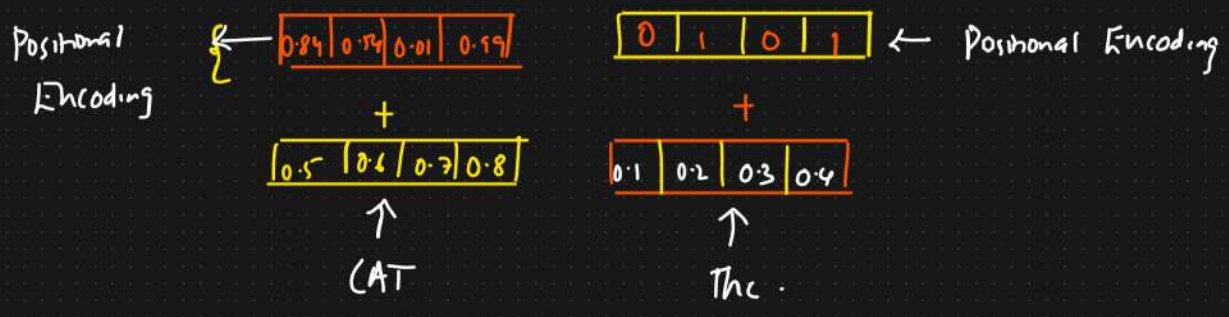
$$P.E(1,0) = \sin\left(\frac{1}{10000^{0/4}}\right) = \sin(1) = 0.8415$$

$$PE(1,1) = \cos\left(\frac{1}{10000^{2/4}}\right) \approx 0.5403$$

$$PE(1,2) = \sin\left(\frac{1}{10000^{2/4}}\right) \approx 0.01$$

$$PE(1,3) = \cos \approx 0.99995$$

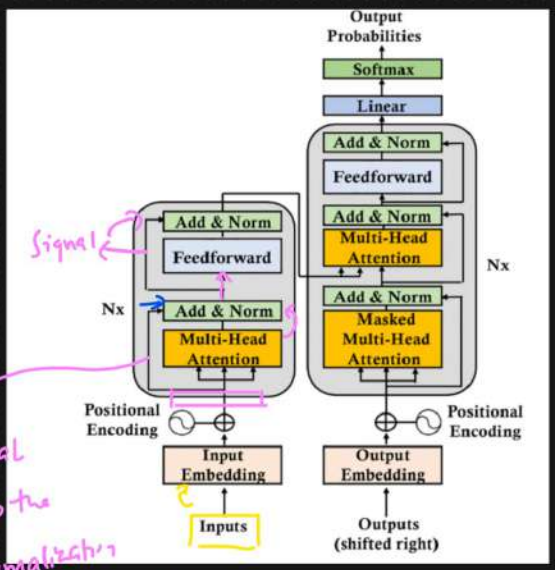




⑨ Layer Normalization In Transformers

Transformers

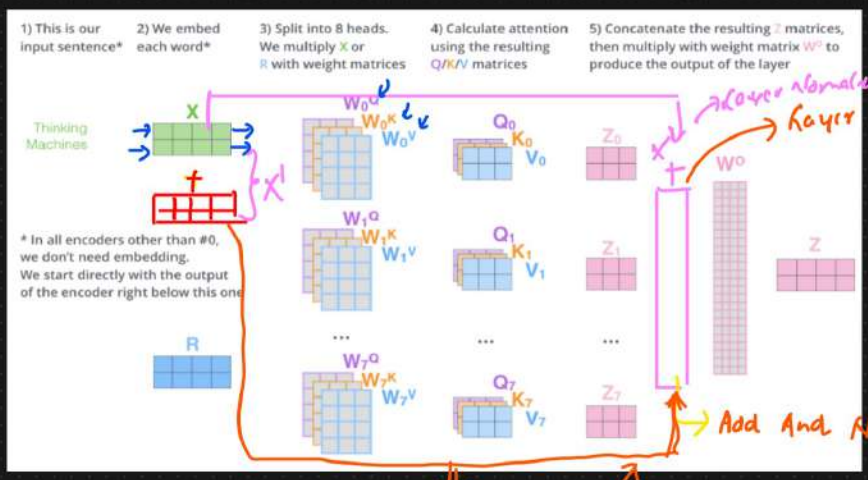
Residuals



Residuals
↳ Additional Signal to the Layer Normalization.

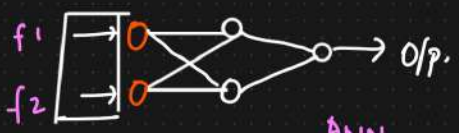
① Self Attention layer
② Multi head Attention
③ Positional Encoding
④ Layer Normalization

ADD AND Normalize



Normalization

→ Batch Normalization
→ Layer Normalization



f1 House Size	f2 No of Rooms	Price
1200	2	45
1500	3	70

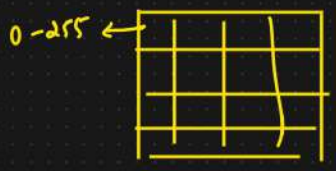
Normalization : Standard Scaling

μ, σ ← | 2000 | 3.5 | 80

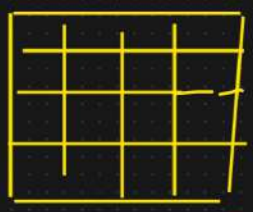
$$Z_{score} = \frac{x_i - \mu}{\sigma} \Rightarrow \boxed{\mu = 0, \sigma = 1}$$

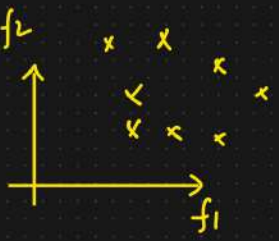
$f_1 \longrightarrow f_1' \Leftarrow \mu = 0, \sigma = 1$

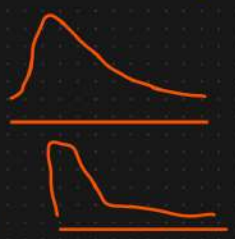
Deep learning : I/p Image ⇒ Min Max Scaler

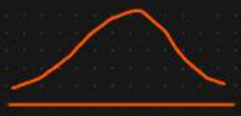


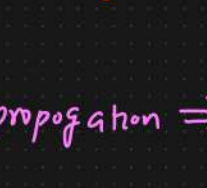
⇒ Min Max Scaler ⇒



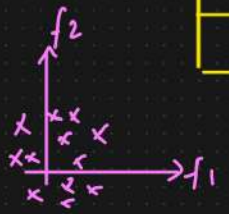








Zero centered

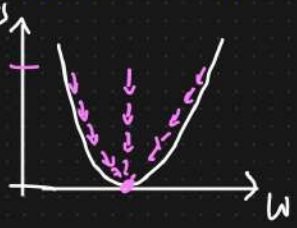


Advantages

① Improved Training Stability ⇓ Vanishing and exploding Gradient problem

$\{\mu = 0, \sigma = 1\}$ $\mu = 0, \sigma = 1$

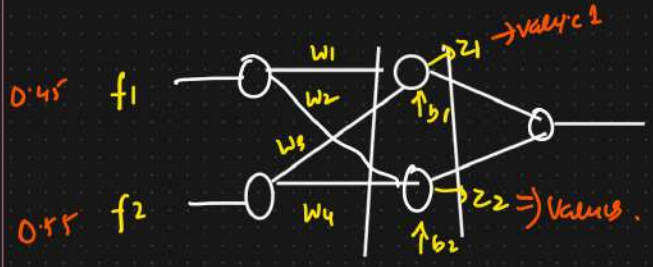
② Faster Convergence



⇒ Back propogation ⇒ Stable updates.

{Batch Normalization}

Normalize Normalized



f1 House size	f2 Rooms	Price	Z_1 =	Z_2 =
0.45	0.55	45	—	—
0.60	0.20	—	—	—
—	—	—	—	—

$Z_1 = \sigma[(0.45 * W_1 + 0.55 * W_3) + b_1]$ = Value 1 } Does this distribution

$Z_2 = \sigma($ $+b_2) =$ Value 2 }. $\boxed{\mu = 0, \sigma = 1}$ μ_1, σ_1 μ_2, σ_2

Batch Normalization Vs Layer Normalization

f1	f2	Z_1	Z_2	
—	—	—	—	→ μ_1, σ_1
—	—	—	—	→ $\mu_2 \sigma_2$
—	—	—	—	→ $\mu_3 \sigma_3$

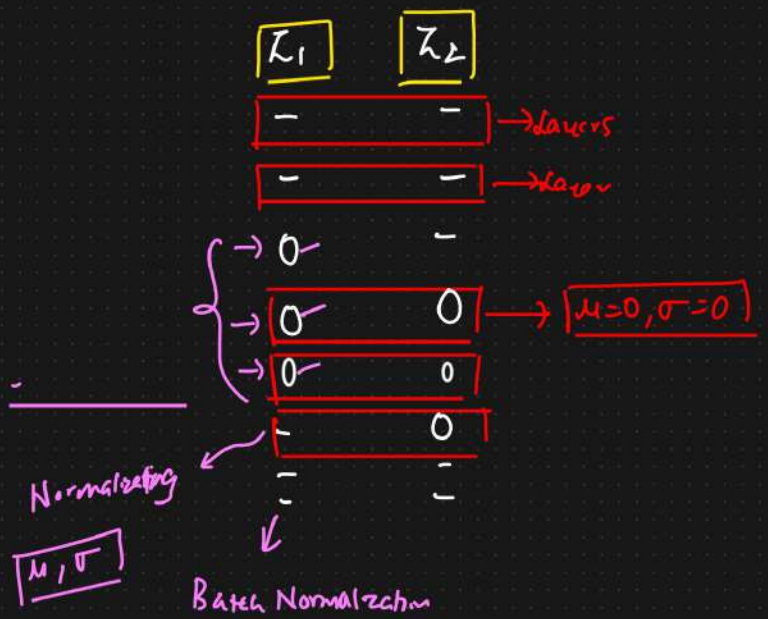
$Z_{score} = \frac{x_i - \mu_1}{\sigma_1}$

$Z_{score} = \frac{x_i - \mu_2}{\sigma_2}$

$Z_{score} = \frac{x_i - \mu_3}{\sigma_3}$ ..

Layer Normalization

$\gamma, \beta \rightarrow$ learnable parameters



Normalization

γ, β

$Z_1 = \sigma[w_1^T x + b_1]$

$Y = \gamma\left[\frac{Z_1 - \mu_1}{\sigma_1}\right] + \beta$

learnable parameters

Scale And Shift parameters

1) "CAT" = [2.0, 4.0, 6.0, 8.0] ← {Vectors} (Normalized)

2) Parameters = γ = [1.0, 1.0, 1.0, 1.0] → Learned Scale

β = [0.0, 0.0, 0.0, 0.0] → Shift

} ⇒ Scale And Shift param

$Z_{score} = \frac{x_i - \mu}{\sigma}$

i) Compute the mean

$\mu = \frac{1}{4}(2.0 + 4.0 + 6.0 + 8.0)$

$= \frac{20.0}{4} = 5.0$

ii) Compute the variance (σ^2)

$\sigma^2 = \frac{1}{4}\left[(2.0-5.0)^2 + (4.0-5.0)^2 + (6.0-5.0)^2 + (8.0-5.0)^2\right] = 5.0$

iii) Normalize the i/p

$\hat{x_i} = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}}$ $\epsilon = 1e^{-5}$ ⇒ Avoid division by 0

$\sqrt{\sigma^2 + \epsilon} = \sqrt{5.0 + 1e-5} \approx \sqrt{5.00001} = 2.236$

$$\hat{x}_1 = \frac{2.0 - 5.0}{2.236} \approx -1.34$$

$$\hat{x}_2 = \frac{4.0 - 5.0}{2.236} \approx -0.45$$

$$\hat{x}_3 = \frac{6.0 - 5.0}{2.236} \approx 0.45$$

$$\hat{x}_4 = \frac{8.0 - 5.0}{2.236} \approx 1.34.$$

Normalized vector

$$\hat{x} = [-1.34, -0.45, 0.45, 1.34]$$

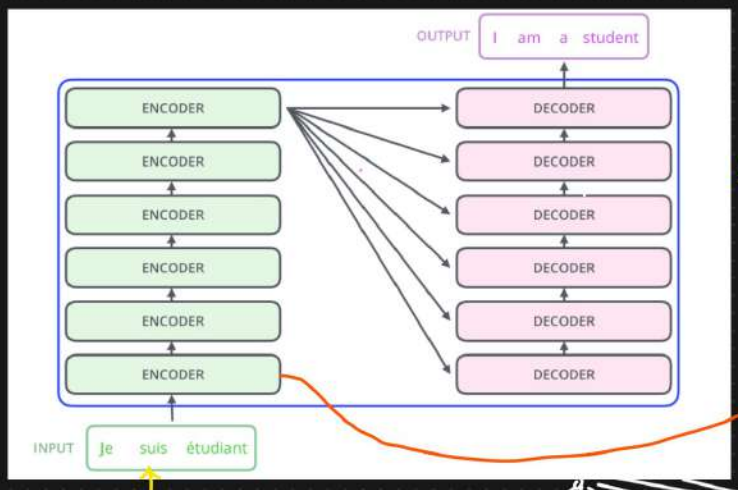
4) Scale And Shift

$$\gamma = [1.0, 1.0, 1.0, 1.0] \quad \beta = [0.0, 0.0, 0.0, 0.0]$$

$$y_i = \gamma_i \hat{x}_i + \beta_i$$

$$y = [-1.34, -0.45, 0.45, 1.34].$$

⑥ Encoder Architecture [Research Paper]



Sequence to Sequence

(Complex)

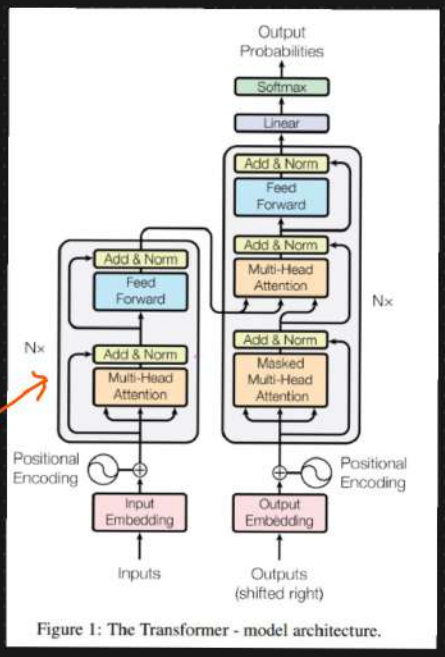


Figure 1: The Transformer - model architecture.

Feed Forward NN. → ○○○○○○○○○

(Add And Norm) → • • • • •

Multi Head Attention → $Z_1, Z_2, Z_3, Z_4, Z_5, Z_6, Z_7, Z_8$ {Research paper}.

Residuals

Text Embeddings + Positional Encoding ⇒ 512. → {Research paper}.

I/P Sequence

Every word = 512.

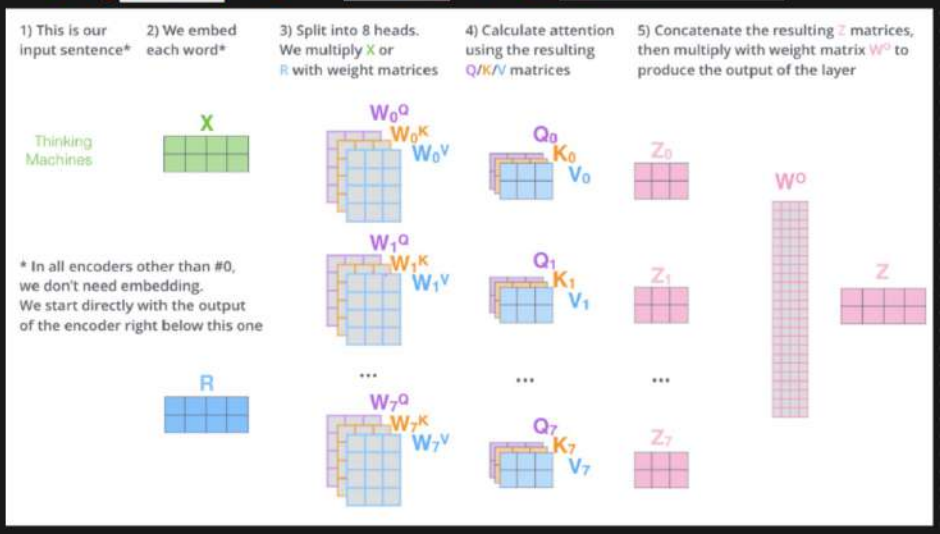
$Q_1 = 64$

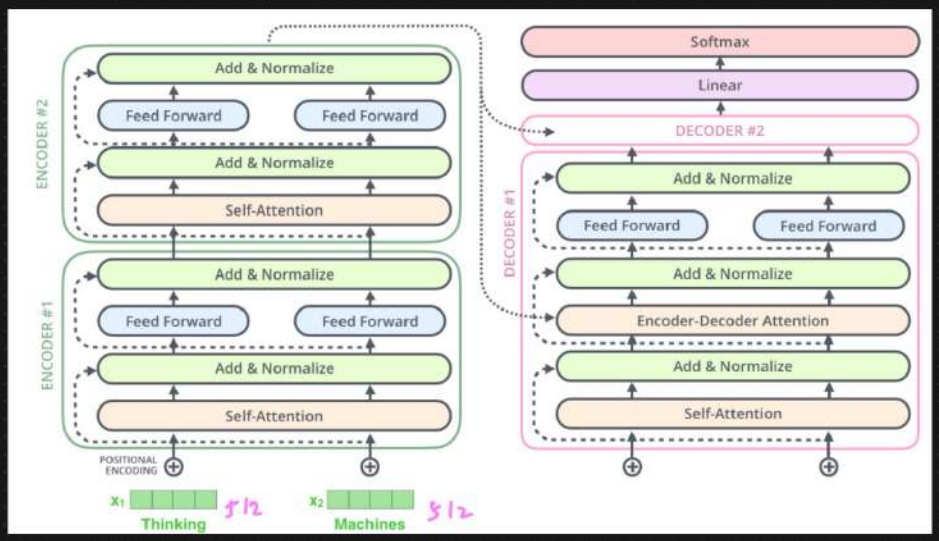
$K = 64$

$V = 64$

$\sqrt{64} = 8$ //.

Self Attention to Feed Forward Neural N/W





Embedding Vector: 512

Q, K, V = 64

Head Attention: 8

① Residual connection : Skip connection NN.

1) Addressing the Vanishing Gradient Problem

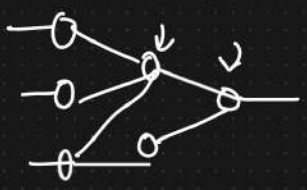
Residuals : Residual connection create a short paths for gradients to flow directly through the n/w. Gradient remains sufficiently large.

2) Improve Gradient flow

Convergence will be faster.

3) Enables Training of Deeper Networks.

④ Feed Forward NN



linear function problem

{Non linear functions}.

① Adding Non linearity

② Processing Each position Independently.

Self Attention → (apture relationships)

FFN → Each token representation Independently.

↓

Transforming these representation further

and allows the model to learn

Richer Representation.

} ANN ⇒

③ FFN → Deeper ⇒ Adds Depth to the Model.

Depth ↑↑ ⇒ More learnings → DATA

⑤ Decoders In Transformers

3 main Components

The transformer decoder is responsible for generating the output sequence one token at a time, using the encoder's output and the previously generated tokens.

① Masked Multi Head Self Attention ✓

② Multi Head Attention (Encoder Decoder Attention)

③ Feed Forward Neural Network.

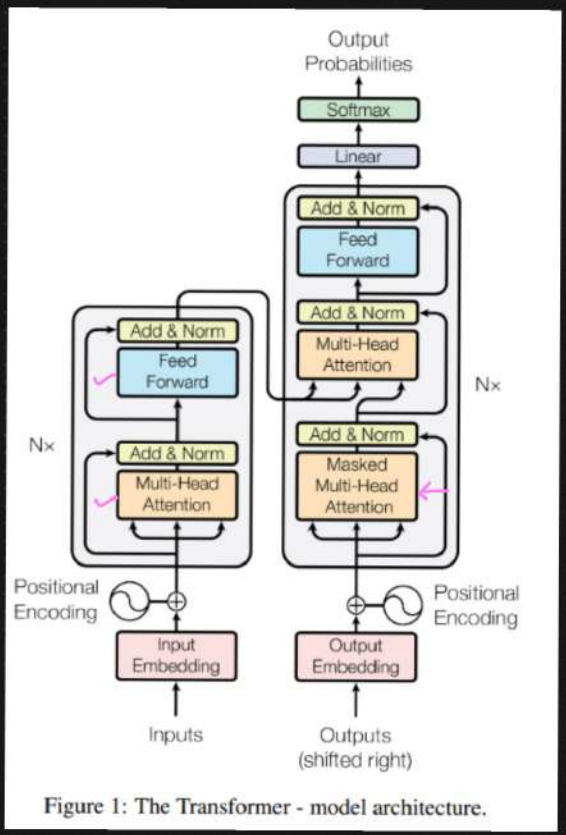
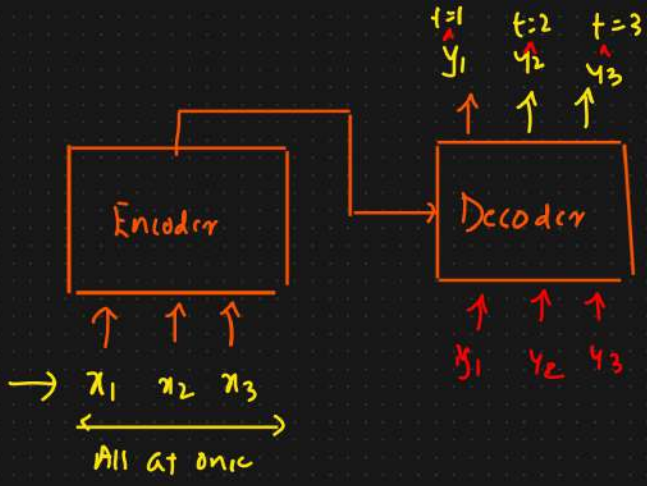


Figure 1: The Transformer - model architecture.



$\langle y_1, y_2, y_3 \rangle$

① Training Mechanism

② Inference Mechanism

⑧ Masked Multi Head Self Attention

① I/P Embedding And Positional Embedding ✓ → Zero padding → Sequence length Equal

② Linear Projection for Q, K, V

③ Scaled Dot Product Attention

✓④ Mask Application } ⇒ Try to understand the imp.

⑤ Multi Head Attention

⑥ Concatenation And Final Linear Projection

⑦ Residual connection And Layer Normalization

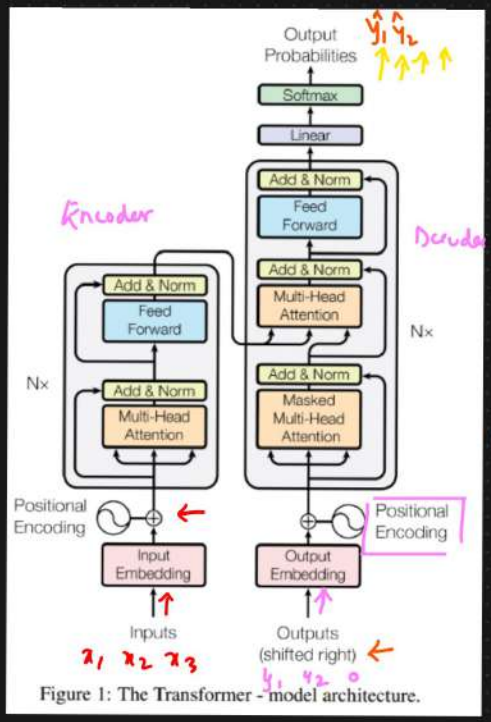


Figure 1: The Transformer - model architecture.

Dataset

Eng: $\langle x_1, x_2, x_3 \rangle$

Hindi: $\langle y_1, y_2 \rangle$ ← $\langle y_1, y_2, 0 \rangle$ ← Zero padding

① Linear Projections Q, K, V

② Scaled Dot Product Attention

③ Mask Application → Look Ahead Mask
→ Padding Mask

Masked Multi Head Attention.

I/P [4 5 6 7] O/P [1 2 3]

[1 2 3 0]

↓

4 dimension vector

i) Input Embedding and Positional Encoding

Output Embedding [STEP 1]

$$\begin{bmatrix} [0.1, 0.2, 0.3, 0.4], \\ [0.5, 0.6, 0.7, 0.8], \\ [0.9, 1.0, 1.1, 1.2], \\ [0.0 \quad 0.0 \quad 0.0 \quad 0.0] \end{bmatrix} + PE = 0$$

Step 2: Linear Projection for Q, K, and V.

$W_Q = W_K = W_V = I$

Create query (Q), Key(K) and value(V) vectors

Q = Output Embedding * W_Q = Output Embedding

K = " * W_K = "

V = " * W_V = "

$$Q = K = V = \begin{bmatrix} [0.1, 0.2, 0.3, 0.4], \\ [0.5, 0.6, 0.7, 0.8], \\ [0.9, 1.0, 1.1, 1.2], \\ [0.0 \quad 0.0 \quad 0.0 \quad 0.0] \end{bmatrix} \begin{bmatrix} 0.1 \\ 0.2 \\ 0.3 \\ 0.4 \end{bmatrix}^T \begin{bmatrix} 0.5 \\ 0.6 \\ 0.7 \\ 0.8 \end{bmatrix} \cdot \begin{bmatrix} 0.9 \\ 1.0 \\ 1.1 \\ 1.2 \end{bmatrix} \begin{bmatrix} 0 \\ 0 \\ 0 \\ 0 \end{bmatrix}$$

③ Scaled Dot Product Attention Calculation

$$\text{Scores} = Q * K^T / \sqrt{dk}$$

$$= Q * K^T / 2.$$

$$\begin{bmatrix} [0.1*0.1 + 0.2*0.2 + 0.3*0.3 + 0.4*0.4, \\ 0.1*0.5 + 0.2*0.6 + 0.3*0.7 + 0.4*0.8, \\ 0.1*0.9 + 0.2*1.0 + 0.3*1.1 + 0.4*1.2 \\ 0.1*0 + 0.2*0 + 0.3*0 + 0.4*0] \end{bmatrix}$$

Scores : [[0.3, 0.7, 1.1, 0.0]
[0.7, 1.9, 3.1, 0.0]
[1.1 3.1 5.1 0.0]
[0.0 0.0 0.0 0.0]]

[ ]
[ ]
[ ]]

⑧ Masked Application

It helps manage the structure of the sequences being procesed And ensures the models behaves correctly during training And Inferencing

Reasons

① Handling Variable length Sequences with Padding MASK

Purpose

① To handle sequences of different length in batch

② To ensure that padding tokens, which are added to make sequences of uniform length, do not affect the model prediction

Eg: i/p ← Sequence 1 [1, 2, 3]

o/p ← Sequence 2 [4, 5, 0] 0 is the padding token

→ 100.

↑ → Influence the Attention Mechanism

⇓

lead to Incorrect or biased predictions.

o/p

→ 41 42 0 0 0 0 0

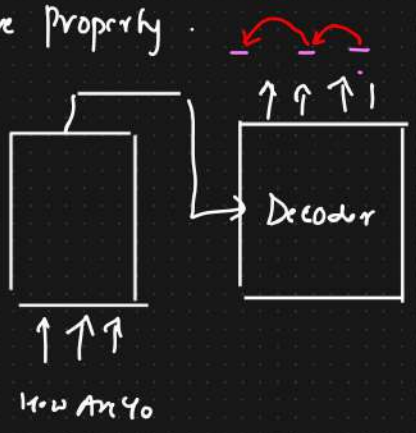
A padding mask ⇒ The tokens Are ignored.

Masking → Padding Mask
→ Look Ahead Mask. }

Padding Mask [1 1 1]
[1 1 0]

② Look Ahead Mask → Maintain Auto Regressive Property.

① To ensure that each position in the decoder output sequence can only attend to the previous position, but no future position



② Sequence → Language Modelling, Translation

Eg: [4, 5, 0] → [1, 1, 0] → 1D Mask.

Convert 1D to 2D Mask

For each token in the sequence, the mask should indicate which tokens it can attend to.

Token 1 attends to token 1, 2

Attention Mechanism ⇐

$$\begin{bmatrix} 1 & 1 & 0 \\ 1 & 1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

1, 2

③ Look Ahead Mask → Decoder output

$$\begin{bmatrix} [1 & 0 & 0] \\ [1 & 1 & 0] \\ [1 & 1 & 1] \end{bmatrix}$$

④ Combine Padding And Looking Ahead Mask

Element Wise multiplication of 2 mask

$$\text{Combine Mask} = \begin{bmatrix} [1, 0, 0] \\ [1, 1, 0] \\ [0, 0, 0] \end{bmatrix}$$

④ Mask

Scores : $\begin{bmatrix} [0.3, 0.7, 1.1, 0.0] \\ [0.7, 1.9, 3.1, 0.0] \\ [1.1, 3.1, 5.1, 0.0] \\ [0.0, 0.0, 0.0, 0.0] \end{bmatrix}$

Look Ahead Mask

$\begin{bmatrix} [1, 0, 0, 0] \\ [1, 1, 0, 0] \\ [1, 1, 1, 0] \\ [1, 1, 1, 1] \end{bmatrix}$

Padding Masking [extended to 2D Format]

$\begin{bmatrix} [1, 1, 1, 0] \\ [1, 1, 1, 0] \\ [1, 1, 1, 0] \\ [0, 0, 0, 0] \end{bmatrix}$

Combined Mask : Look Ahead Mask * Padding Mask

$\begin{bmatrix} [1*1, 0*1, 0*1, 0*0] \\ [1*1, 1*1, 0*1, 0*0] \\ [1*1, 1*1, 1*1, 0*0] \\ [1*1, 1*1, 1*1, 1*0] \end{bmatrix} = \begin{bmatrix} [1, 0, 0, 0], \\ [1, 1, 0, 0], \\ [1, 1, 1, 0], \\ [1, 1, 1, 0] \end{bmatrix} \Rightarrow \begin{bmatrix} [1, -\infty, -\infty, -\infty], \\ [1, 1, -\infty, -\infty], \\ [1, 1, 1, -\infty] \\ [1, 1, 1, -\infty] \end{bmatrix}$

Masked Score

$\begin{bmatrix} [0.3, -\infty, -\infty, -\infty] \\ [0.7, 1.9, -\infty, -\infty] \\ [1.1, 3.1, 5.1, -\infty] \\ [0.0, 0.0, 0.0, -\infty] \end{bmatrix}$

→ 0

Zero out the influence when the Softmax is applied.

Attention weights.

⑧ Softmax

Softmax Score = Softmax (Masked Scores)

$$= [[1.0, 0.0, 0.0, 0.0],$$
$$[0.3, 0.7, 0.0, 0.0],$$
$$[0.1, 0.3, 0.6, 0.0],$$
$$[1.0, 0.0, 0.0, 0.0]]$$

⑨ Weigth Sum of Value

Attention O/P = Softmax Scores * V.

Masking

Masking in the transformer architecture is essential for several reasons. It helps manage the structure of the sequences being processed and ensures the model behaves correctly during training and inference. Here are the key reasons for using masking:

1. Handling Variable-Length Sequences with Padding Mask

Purpose
To handle sequences of different lengths in a batch.
To ensure that padding tokens, which are added to make sequences of uniform length, do not affect the model's predictions.

2. Maintaining Autoregressive Property with Look-Ahead Mask

Purpose
To ensure that each position in the decoder output sequence can only attend to previous positions and itself, but not future positions.
This is crucial for sequence generation tasks like language modeling and translation, where the model should not have access to future tokens when predicting the current token.

④ Encoder Decoder Multi Head Attention

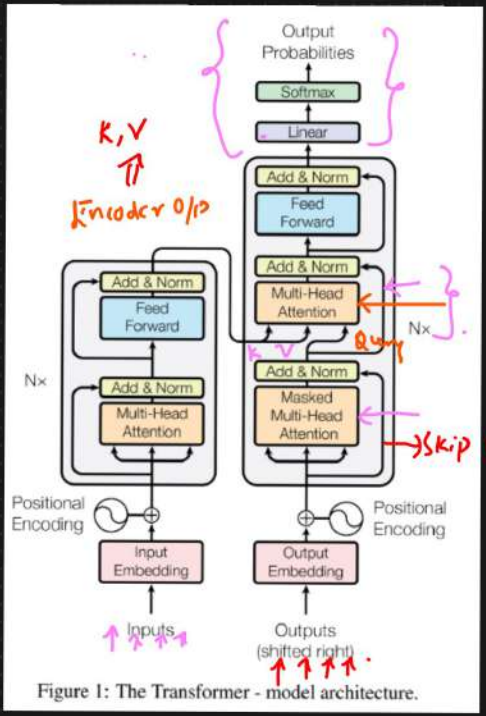


Figure 1: The Transformer - model architecture.

① Encoder O/P → Set of Attention vector K & V

② Masked Multi Head → Attention vector Q {Query Vector}

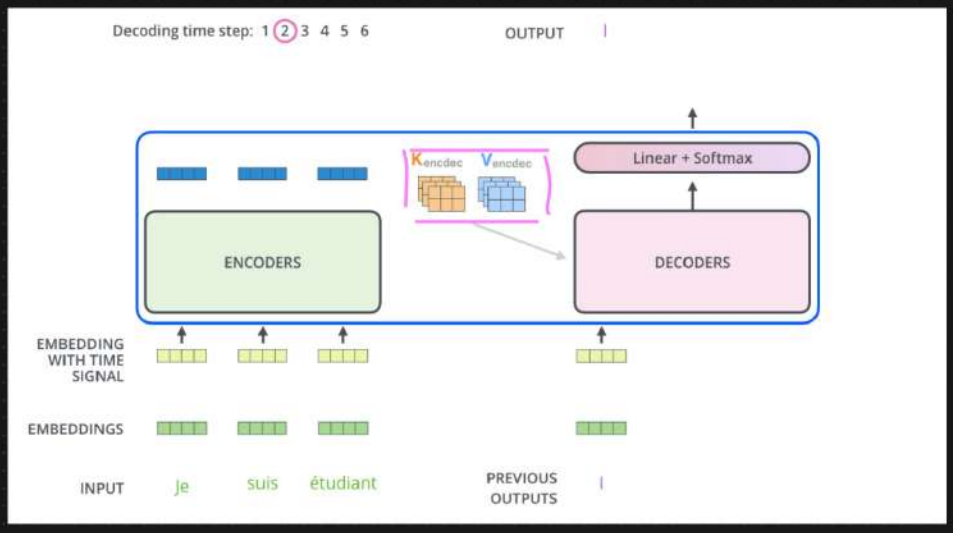
These are to be used by each decoder in its "encoder – decoder" attention layer

⇓

Helps the Decoder to focus on appropriate places in the i/p sequence

Dataset

i/p $\langle x_1, x_2, x_3 \rangle$ O/p $\langle y_1, y_2, y_3 \rangle$.



④ The Final Linear And Softmax Layer

{Vectors ⟶ o/p word}

{BLOG} ⇒ Transformers

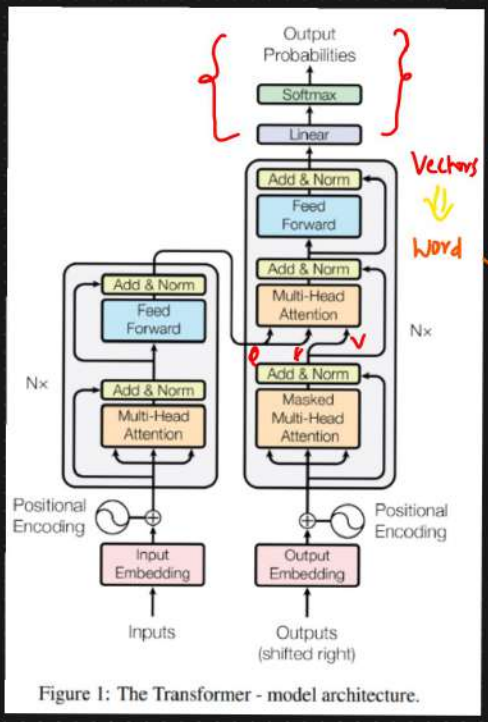
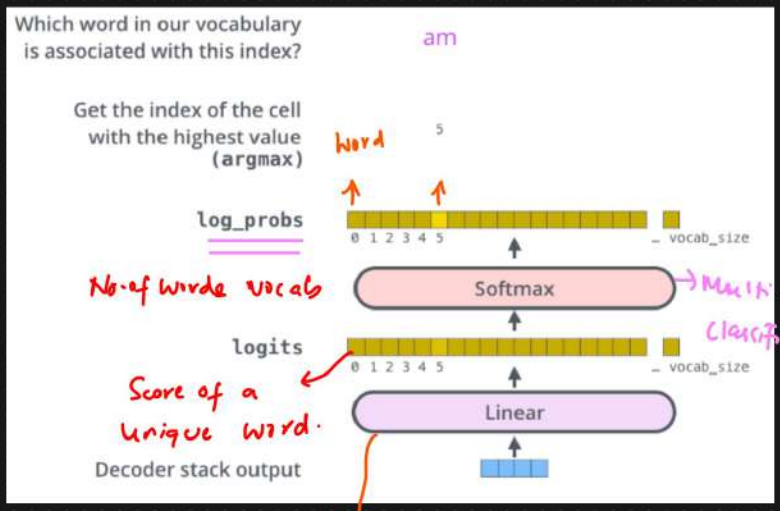


Figure 1: The Transformer - model architecture.



Linear ⇒ The linear layer is a simple fully connected neural n/w that projects the vector produced by the stack of Decoder ⇒ logits vector

⇒

Model ⇒ 10,000 ⇒ vocabulary ⇒ logits vector = 10000 cells wide

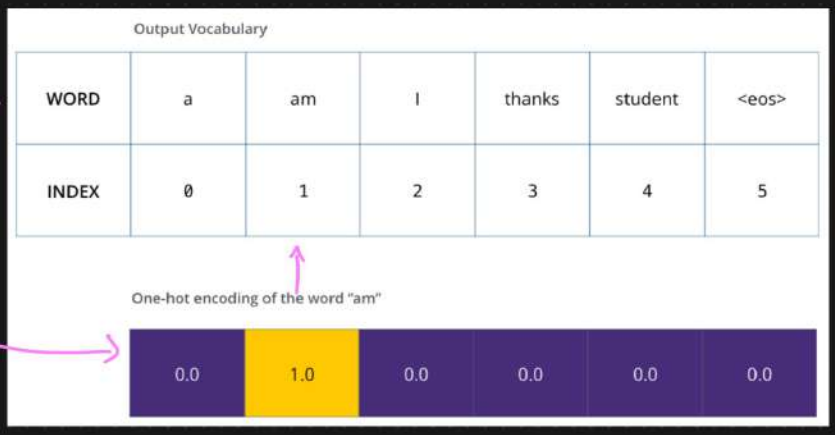
② Softmax layer turns those scores into probabilities (all add upto 1.0).

The cell with the highest probability is chosen, and the word associated with it is produced as the o/p. ⇒ time stamp.

Recap of Training

Output Vocabulary

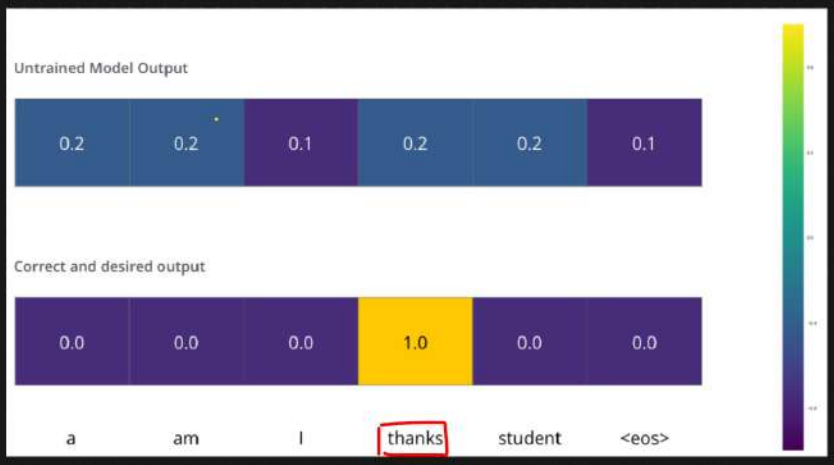
WORD	a	am	I	thanks	student	<eos>
INDEX	0	1	2	3	4	5



OHE ←

Merci → Thanks

I am a student <eos>.



⇒ Loss function ↓↓.

Back Propogation

i/p

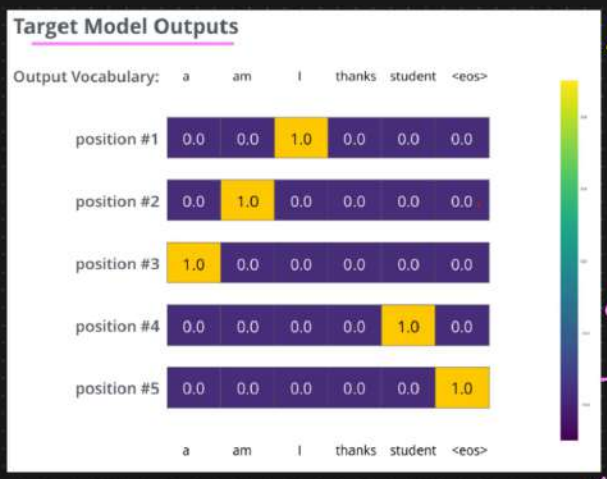
a, am, i, student

o/p

I am a student

↓ ↓ ↓ ↓

OHE OHE OHE OHE



Loss